



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Aidea

Follow up audit

August 2026

Prepared by

naman

Varun_Sharma

0xAlix2

Introduction

A time-boxed security review of the **Aidea** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A code review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About Aidea

Aidea is a launchpad template pairing an ERC20 token with an auto-evolving ERC721 identity NFT on Base. A user's first buy from the Uniswap V2 pair auto-mints a one-per-wallet NFT whose six on-chain traits (Background, Sigil, Weapon, Character, Border, Effect) evolve with trading behavior based on their entry time, holding duration, balance growth, etc. Holders who reach the final Character trait are eligible to claim a one-time, balance-doubling reward.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

Scope

The following folders and every file in them were in scope of the audit:

- `Aidea/tree/main/template_contracts.nft.sol`
- `Aidea/tree/main/template_contracts.token.sol`
- `Aidea/tree/main/template_contracts.traits.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 2 issues
- Medium: 7 issues
- Low and Informational: 15 issues

Findings Summary

ID	Title	Severity	Status
[H-01]	Reward eligibility reads duration and size from unrelated state, letting one position drain the entire pot	High	Fixed
[H-02]	Anyone can reset a holder's days held clock with a zero value transfer and deny their reward	High	Fixed
[M-01]	<code>computeWeaponValue</code> transposes two tiers and applies a floor 2.5x stricter than documented	Medium	Fixed
[M-02]	Growth traits are measured against a baseline the holder chooses, so the rarest tiers cost nothing	Medium	Fixed
[M-03]	Any contract can be flagged a forwarder and stripped of its identity NFT without consent	Medium	Fixed
[M-04]	Reward claims do not refresh balance-based NFT traits	Medium	Fixed
[M-05]	Attacker Can Corrupt A Victim's Trading Stats Using A Stale Forwarder	Medium	Fixed
[M-06]	Forwarder-hop <code>lastBuyStamp</code> overwrite uses the forwarder's own buy timestamp instead of <code>block.timestamp</code> , letting to's holding-streak clock silently move backward	Medium	Fixed
[M-07]	Forwarder's small-transfer threshold uses a self-resetting baseline, letting chunked transfers drain a forwarder without triggering identity resolution	Medium	Fixed
[L-01]	A one wei payment permanently forfeits CHARACTER5, a trait defined as "hasn't sold"	Low	Fixed
[L-02]	Anyone can force-mint an identity NFT onto any address, permanently blocking every identity transfer into it	Low	Fixed
[L-03]	<code>getDaysHeld</code> and <code>getDaysHeldInHours</code> implement the same rule with different comparison operators	Low	Fixed
[L-04]	<code>supportsInterface</code> advertises ERC-4906 but <code>MetadataUpdate</code> is never declared or emitted	Low	Fixed

ID	Title	Severity	Status
[L-05]	<code>totalSupply</code> returns a mint counter that is never decremented when an NFT is burned	Low	Fixed
[L-06]	<code>Border</code> and <code>Background</code> trait values do not increase with rarity	Low	Fixed
[L-07]	Redundant <code>>= 90 days</code> branch in <code>computeBackgroundValue</code> contradicts its own comment	Low	Fixed
[L-08]	The Uniswap pair can be made a reward claimant and its payout is extractable by anyone	Low	Fixed
[L-09]	Forwarder-delivered buys preserve <code>sellStreak</code> while incrementing <code>buyStreak</code>	Low	Fixed
[L-10]	Any User Can Fabricate Their Own Trading History For Free	Low	Fixed
[L-11]	Any User Can Erase Their Own Effect Trait With A Dust Buy	Low	Fixed
[L-12]	SpeedMultiplier Inconsistently Accelerates Trait Thresholds	Low	Acknowledged
[L-13]	QuoteTokensInWETH Returns A Stale, Uncorrectable Price Quote	Low	Fixed
[L-14]	<code>isCharacter6</code> incorrectly checks for token to be held for more than 90 days	Low	Fixed
[I-01]	Transfers to the zero address do not reduce <code>totalSupply</code>	Informational	Fixed

Detailed Findings

[H-01] Reward eligibility reads duration and size from unrelated state, letting one position drain the entire pot

Severity

Impact:

High

Likelihood:

Medium

Description

`claimReward` requires two things of a holder: that they have held for 90 days, and that they hold at least 0.1% of supply. The duration comes from `firstBuyStamp` through `getDaysHeld`, and the size comes from `lastTokenAmount`. Nothing ties the two together, and each can be established independently.

The duration is free. `pair.sync()` followed by `pair.skim(W)` is permissionless and sends a zero value transfer from the pair, which stamps `firstBuyStamp`, leaves `lastSellStamp` at zero and mints `W` an identity NFT. `W` holds no tokens and risks no capital, yet its clock is now running.

The size is grafted on at the last moment. Any ordinary incoming transfer refreshes the recipient's recorded size and nothing else:

```
updateUserStatsOnSell(from, uint128(value));
userStats[to].lastTokenAmount = uint128(balances[to]);
```

`firstBuyStamp` and `lastSellStamp` are untouched, so a balance handed over on day 91 inherits a 91 day old clock it never earned.

Because `hasClaimedReward` is keyed per tokenId, every pre-aged wallet carries its own unused claim slot. One position can therefore be walked through many of them. With a 10,000,000e18 supply the per claim cap is `totalSupply / 400`, or 25,000e18, so forty pre-aged wallets and a single 25,000e18 position drain the full 1,000,000e18 pot. The attacker's own position is never at risk and is returned in full at the end.

One practical constraint is worth stating. `claimReward` is restricted to the owner, and `require(balances[nftOwner] >= tokensHeld)` means the position must still sit in the wallet when the call is made, so the sequence needs one owner call per wallet with the position parked there. Every one of those wallets nonetheless presents as a legitimate long term holder on chain, so an owner processing claims normally walks the whole sequence.

Proof of Concept

Foundry test, passing. `POT` is 1,000,000e18 and `MAX_REWARD` is `TOTAL_SUPPLY / 400`.

```
function test_C01_RotatePreAgedWalletsDrainsPot() public {
    uint256 N = POT / MAX_REWARD; // 40 claims empty the pot exactly

    // 1) Pre-age N identity wallets for FREE via permissionless zero-value
    skim.
    // Each gets firstBuyStamp = now, lastSellStamp = 0, and an identity
    NFT, while
    // holding ZERO tokens and risking ZERO capital.
    address[] memory w = new address[](N);

    for (uint256 i = 0; i < N; i++) {
        w[i] = makeAddr(string.concat("aged", vm.toString(i)));
        pair.skim(w[i]);
    }
}
```

```

        assertEq(nft.balanceOf(w[i]), 1, "identity force-minted");
        assertEq(token.getUserStats(w[i]).firstBuyStamp, block.timestamp,
"clock started");
        assertEq(token.getUserStats(w[i]).lastSellStamp, 0, "receive-only
=> never sold");
    }

    // 2) Let the identities age. No position is held during this window.
    vm.warp(block.timestamp + 91 days);

    // 3) Seed ONE real position (0.25% of supply) into the first aged
wallet. A plain
    // transfer refreshes only lastTokenAmount; the 91-day-old clock is
untouched.
    token.transfer(w[0], MAX_REWARD);

    assertEq(token.rewardPot(), POT, "pot full before attack");

    // 4) Rotate the single position through every pre-aged wallet,
claiming once each.
    for (uint256 i = 0; i < N; i++) {
        // Owner processes what looks like a perfectly eligible claim.
        token.claimReward(nft.userToTokenId(w[i]));

        if (i + 1 < N) {
            uint256 bal = token.balanceOf(w[i]);
            vm.prank(w[i]);
            token.transfer(w[i + 1], bal);
        }
    }

    // 5) The entire escrowed pot has been drained.
    assertEq(token.rewardPot(), 0, "reward pot fully drained");
    assertEq(token.balanceOf(w[N - 1]), MAX_REWARD + POT, "attacker: seed +
entire pot");
}

```

Recommendations

Make the duration and the size describe the same position. Record when a wallet's balance first rose above the eligibility threshold and require that it has stayed above it continuously since, rather than measuring against a timestamp that any third party can set for free on a wallet holding nothing.

Refusing to stamp `firstBuyStamp` on a zero value receipt narrows the cheapest entry point but does not fix this on its own, since the timestamp and the balance would still be independent.

[H-02] Anyone can reset a holder's days held clock with a zero value transfer and deny their reward

Severity

Impact:

Medium

Likelihood:

High

Description

`Token.triggerUpdate` decides that a transfer is a buy based on where the tokens came from, never on who asked for them. Any transfer whose sender is the pair reaches:

```
function updateUserStatsOnBuy(address user, uint128 amount) internal {
    UserStats storage stats = userStats[user];
    stats.lastBuyStamp = uint128(block.timestamp);
    ...
}
```

`UniswapV2Pair.skim(to)` takes an arbitrary `to` and is callable by anyone, and on a synced pool the surplus is zero. So `pair.sync()` followed by `pair.skim(victim)` moves no tokens, costs the caller only gas, and rewrites that victim's `lastBuyStamp` to the current time.

Whether that harms the victim depends on which clock `getDaysHeld` uses for them. A holder who has never sold is measured from `firstBuyStamp` and is unaffected. Once a holder has any non zero `lastSellStamp`, the measurement switches to `lastBuyStamp`, and moving that stamp to now collapses their days held to zero.

The consequence is immediate and needs no further action from the attacker. `isCharacter6` fails, so `claimReward` reverts with `holder does not have the final character trait` and the holder is denied up to 25,000e18, and their Sigil trait drops to `NONE`. Nothing about their actual position changed.

The exposed population is far wider than people who have sold on the market. An ordinary wallet to wallet transfer is recorded as a sell, so a holder who once paid someone a single wei is moved onto the `lastBuyStamp` clock permanently and stays exposed from then on.

The attack is repeatable for dust, so a victim who ages back into eligibility can simply be reset again, indefinitely. It is also asymmetric in a way that invites abuse: an attacker who never sells keeps the `firstBuyStamp` clock, remains immune themselves, and can deny every other holder's claim while their own stays valid.

Proof of Concept

Foundry tests, both passing. The first establishes that the victim is genuinely entitled to the reward, the second denies that same reward with a single zero value call.

```
/// Build a holder that has sold (so its clock runs off lastBuyStamp), then
re-bought
/// and held 90 days, making it eligible for the reward.
```

```

function _makeEligibleVictim() internal returns (address victim, uint256
tokenId) {
    victim = makeAddr("victim");
    address friend = makeAddr("friend");

    // Day 0: buy 40k from the pair -> identity minted, lastSellStamp still
0.
    pair.swapTo(victim, 40_000e18);

    // An ordinary peer transfer marks the victim as having "sold":
lastSellStamp = now.
    vm.prank(victim);
    token.transfer(friend, 1);
    assertGt(token.getUserStats(victim).lastSellStamp, 0, "lastSellStamp
set by a plain transfer");

    // Next day: re-buy -> lastBuyStamp > lastSellStamp, clock now runs off
lastBuyStamp.
    vm.warp(block.timestamp + 1 days);
    pair.swapTo(victim, 1_000e18);

    // Hold 90 days -> eligible.
    vm.warp(block.timestamp + 90 days);
    tokenId = nft.userToTokenId(victim);
}

/// Baseline: the victim is genuinely eligible and the owner can pay the
reward.
function test_H02_EligibleVictimCanClaim() public {
    (address victim, uint256 tokenId) = _makeEligibleVictim();

    uint256 potBefore = token.rewardPot();
    uint256 balBefore = token.balanceOf(victim);

    token.claimReward(tokenId);

    assertEq(token.rewardPot(), potBefore - 25_000e18, "reward paid out");
    assertEq(token.balanceOf(victim), balBefore + 25_000e18, "victim
received the reward");
}

/// Attack: a dust skim resets the clock, denying the same reward.
function test_H02_DustSkimResetsClockAndDeniesReward() public {
    (address victim, uint256 tokenId) = _makeEligibleVictim();

    uint128 lastBuyBefore = token.getUserStats(victim).lastBuyStamp;

    // Attacker resets the victim's clock with a permissionless zero-value
skim.
    pair.sync();
    vm.prank(attacker);
    pair.skim(victim);

    assertGt(token.getUserStats(victim).lastBuyStamp, lastBuyBefore,

```

```

"lastBuyStamp moved to now");

    // The reward is now denied, even though nothing about the position
    changed.
    vm.expectRevert("holder does not have the final character trait");
    token.claimReward(tokenId);
}

```

Recommendations

Only write `userStats[to]` when `to` is the account that actually initiated the swap, or is a beneficiary that explicitly registered for it. A pair originated transfer to an address that did not ask for it should not be treated as that address making a buy, and no third party should be able to move another holder's `lastBuyStamp` in either direction.

[M-01] `computeWeaponValue` transposes two tiers and applies a floor 2.5x stricter than documented

Severity

Impact:

Medium

Likelihood:

Medium

Description

`computeWeaponValue` contains two independent errors.

```

//Below 0.01%, no weapon
if (lastTokenAmount * 100000 < totalSupply * 25) return Weapon.NONE;

//Weapon is a user that has doubled their initial buy amount
if (lastTokenAmount >= firstBuyAmount * 2) {
    return Weapon.WEAPON3;
} else if (lastTokenAmount >= (firstBuyAmount * 150) / 100) {
    return Weapon.WEAPON1;
} else if (lastTokenAmount >= (firstBuyAmount * 125) / 100) {
    return Weapon.WEAPON2;
}

```

The floor is `25 / 100000`, which is 0.025%, while the comment above it says 0.01% and `isCharacter3` uses `currentTokensHeld * 10000 >= totalSupply`, which is 0.01%. To match either of those the multiplier would have to be `10`. On the `10,000,000e18` supply the coded floor is `2,500e18` against an

intended 1,000e18, so a holder between those two figures can earn CHARACTER3 and still be denied every Weapon.

The tier labels are also swapped. `Weapon` is declared as `NONE`, `WEAPON1`, `WEAPON2`, `WEAPON3`, so the values ascend with rank, but 50% growth returns `WEAPON1` while 25% growth returns `WEAPON2`. The holder with the better performance receives the lower tier. The thresholds themselves are ordered correctly, only the two returns are exchanged.

This is not an arbitrary numbering scheme. `WEAPON3` is correctly placed at the highest growth threshold, which is what shows the values are meant to read as ranks and that these two are simply reversed.

Both errors are permanent once deployed. `Traits` is created inside the `Token` constructor with no setter, and these values are hardcoded in a `pure` function.

Proof of Concept

Foundry tests, both passing.

```
/// A holder who grew 50% receives a LOWER weapon tier than one who grew
only 25%.
function test_M01_WeaponTiersAreTransposed() public view {
    uint128 baseline = 100_000e18;

    Traits.Weapon grew50 = traits.computeWeaponValue(baseline, (baseline *
150) / 100, SUPPLY);
    Traits.Weapon grew25 = traits.computeWeaponValue(baseline, (baseline *
125) / 100, SUPPLY);

    assertEquals(uint8(grew50), uint8(Traits.Weapon.WEAPON1), "50% growth
returns WEAPON1");
    assertEquals(uint8(grew25), uint8(Traits.Weapon.WEAPON2), "25% growth
returns WEAPON2");
    assertLt(uint8(grew50), uint8(grew25), "the better holder gets the
lower tier");
}

/// At 0.01% of supply the holder qualifies for CHARACTER3 but is denied
every Weapon,
/// because the weapon floor is actually 0.025%.
function test_M01_WeaponFloorIsStricterThanDocumented() public view {
    uint128 documented = uint128(SUPPLY / 10000); // 0.01% = 1_000e18
    uint128 baseline = documented / 2; // doubled, so growth
alone gives WEAPON3

    assertEquals(
        uint8(traits.computeWeaponValue(baseline, documented, SUPPLY)),
        uint8(Traits.Weapon.NONE),
        "0.01% holder gets no weapon"
    );

    assertTrue(traits.isCharacter3(baseline, documented, SUPPLY), "same
holder qualifies for CHARACTER3");
```

```

uint128 actual = uint128((SUPPLY * 25) / 100000); // 0.025% = 2_500e18

assertEq(
    uint8(trait.computeWeaponValue(actual / 2, actual, SUPPLY)),
    uint8(Traits.Weapon.WEAPON3),
    "weapon only unlocks at 0.025%"
);
}

```

Recommendations

Change the floor multiplier from 25 to 10 so it matches the stated 0.01% and agrees with `isCharacter3`, and exchange the two returns so 125% growth yields `WEAPON1` and 150% growth yields `WEAPON2`.

[M-02] Growth traits are measured against a baseline the holder chooses, so the rarest tiers cost nothing

Severity

Impact:

Medium

Likelihood:

Medium

Description

`Token.updateUserStatsOnBuy` stamps `firstBuyAmount` once, from `balances[user]`, on the holder's first pair originated receipt. The growth traits then measure against it:

```

// computeWeaponValue
if (lastTokenAmount * 100000 < totalSupply * 25) return Weapon.NONE; //
floor: 0.025%
if (lastTokenAmount >= firstBuyAmount * 2) return Weapon.WEAPON3; //
"doubled your buy"

// isCharacter3
currentTokensHeld * 10000 >= totalSupply && currentTokensHeld >=
firstBuyAmount * 2

```

Nothing requires that baseline to be economically meaningful. The absolute floor constrains `lastTokenAmount`, the amount held now, and never touches `firstBuyAmount`. So a holder who

arranges for their first receipt to be 0 or 1 wei satisfies `lastTokenAmount >= firstBuyAmount * 2` with any balance at all.

On a 10,000,000e18 supply the floor is 2,500e18, so take two wallets that both hold an identical 3,000e18 and therefore both clear it. The wallet that bought its 3,000e18 honestly has `firstBuyAmount` of 3,000e18, needs `3,000e18 >= 6,000e18`, and gets `NONE`. The wallet whose first receipt was 0 has `firstBuyAmount` of 0, needs `3,000e18 >= 0`, and gets `WEAPON3`.

The effect is inverted from the intent stated in the code comments: the smaller a holder's real first buy, the easier the top tier. An honest buyer needs to double a genuine position, roughly 6,000e18 plus fees and slippage, to earn what the other wallet gets from an ordinary peer transfer.

The ordering is what makes it work. The baseline must be stamped before any tokens arrive, since `firstBuyAmount` captures the balance at that moment. Funding the wallet first and buying second folds the balance into the baseline and the trick fails.

A zero baseline also disables half the dump detector. `isCharacter2` flags a seller on either of two conditions, and the size branch `currentTokensHeld <= firstBuyAmount / 2` becomes `<= 0`, so it only fires on a complete exit. The time branch `timeSinceFirstBuy <= 72 hours / speedMultiplier` still catches a sell inside the first 72 hours, but a wallet that sells 90% of its position after that window is never marked. Integer division makes a 1 wei baseline behave the same way.

No funds are at risk. `claimReward` gates on `isCharacter6`, which reads only days held and tokens held and never consults `firstBuyAmount`.

Proof of Concept

Foundry test, passing.

```
function test_H05_ZeroValueFirstBuyGrantsWeapon3() public {
    address honest = makeAddr("honest");
    address carol = makeAddr("carol");

    // A legitimate buyer: 3,000e18 bought from the pair as their first
    receipt,
    // so their growth baseline is the real position they paid for.
    pair.swapTo(honest, 3_000e18);
    pair.sync();

    assertEq(token.getUserStats(honest).firstBuyAmount, 3_000e18, "baseline
is the real buy");

    // Carol takes a permissionless zero-value skim FIRST, so her baseline
    is 0.
    pair.skim(carol);

    assertEq(nft.balanceOf(carol), 1, "carol force-minted an identity");
    assertEq(token.getUserStats(carol).firstBuyAmount, 0, "firstBuyAmount
pinned to 0");

    // ...then funds herself with an ordinary peer transfer, never touching
```

```

the market.
    token.transfer(carol, 3_000e18);

    // Identical holdings, both above the 0.025% absolute floor.
    assertEq(token.balanceOf(carol), token.balanceOf(honest), "same
balance");

    Traits.Trait[6] memory h = nft.generateNFTTraits(honest);
    Traits.Trait[6] memory c = nft.generateNFTTraits(carol);

    // trait index 2 == Weapon; WEAPON3 is the top tier.
    assertEq(h[2].traitValue, uint8(Traits.Weapon.NONE), "honest buyer
earns no weapon");
    assertEq(c[2].traitValue, uint8(Traits.Weapon.WEAPON3), "zero baseline
earns the top tier");
}

```

Replacing the skim with a genuine 1 wei buy from the pair gives `firstBuyAmount` of 1 instead of 0, and the result is unchanged: `3,000e18 >= 1 * 2` still holds and carol still receives `WEAPON3`.

Recommendations

Require a minimum economically meaningful first buy before stamping `firstBuyStamp` and `firstBuyAmount` in `Token.updateUserStatsOnBuy`, tied to the same share of `totalSupply` the traits already use for their absolute floor.

Do not rely on a `firstBuyAmount > 0` check. A 1 wei first buy passes it and still yields the top tier.

[M-03] Any contract can be flagged a forwarder and stripped of its identity NFT without consent

Severity

Impact:

Medium

Likelihood:

Medium

Description

`Token.triggerUpdate` marks the recipient of a pair originated transfer as a forwarder based only on whether that address has code:

```

if (to.code.length > 0) {
    isForwarder[to] = true;
    userStats[to].lastTokenAmount = uint128(balances[to]);
}

```

```

}

updateUserStatsOnBuy(to, uint128(value));

```

The recipient is chosen by whoever initiates the transfer, and `UniswapV2Pair.skim(to)` accepts an arbitrary `to` and is callable by anyone. On a synced pool the surplus is zero, so `pair.sync()` followed by `pair.skim(V)` reaches this branch for any contract address `V` at the cost of gas alone. `V` is now flagged, and `updateUserStatsOnBuy` also force mints it an identity NFT it never requested.

The flag then sits there until `V` next sends more than 90 percent of its recorded snapshot in a single transfer, which for a vault, payout splitter or treasury is ordinary operation. That transfer takes the forwarder branch and calls `nftContract.forwarderTransfer(from, to)`, which reaches `super._update(to, tokenId, address(0))` directly, so the ERC721 ownership and approval checks never run. `V` approved nobody. `Token.triggerUpdate` then executes `delete userStats[from]`, erasing its recorded history in the same transaction.

If the payee holds no identity, `V`'s NFT is transferred to them. If the payee already holds one, `V`'s NFT is burned and the payee's own `lastBuyAmount` is overwritten with `V`'s, so an unrelated holder loses recorded history to a payment they merely received. When the payee is itself a contract the flag is set on them too, so it moves downstream.

The victim class is ordinary infrastructure holding the token. Nothing about it opted into this protocol.

Proof of Concept

Foundry test, passing. `Distributor` is a benign contract that holds tokens and pays them out, and never touches NFTs.

```

contract Distributor {
    Token public token;

    constructor(Token _t) { token = _t; }

    function pay(address to, uint256 amt) external { token.transfer(to, amt); }
}

function test_H01_ForcedForwarderSeizesNFTOnPayout() public {
    // 1) Attacker force-flags the distributor, for dust, via
    permissionless skim.
    vm.prank(attacker);
    pair.skim(address(dist));

    assertTrue(token.isForwarder(address(dist)), "force-flagged as forwarder");
    assertEquals(nft.balanceOf(address(dist)), 1, "force-minted an identity");

    uint256 tokenId = nft.userToTokenId(address(dist));

    // The distributor NEVER approved anyone over its NFT.

```

```

    assertEq(nft.getApproved(tokenId), address(0), "no token approval");
    assertFalse(nft.isApprovedForAll(address(dist), attacker), "no operator approval");

    // 2) The distributor does its normal job: pay a user >90% of its
    balance in one tx.
    dist.pay(user, 950_000e18);

    // 3) Its identity NFT was seized by the recipient with zero consent.
    assertEq(nft.balanceOf(address(dist)), 0, "distributor lost its NFT");
    assertEq(nft.ownerOf(tokenId), user, "recipient now owns the
distributor's identity");

    // 4) Its accrued stats were wiped.
    assertEq(token.getUserStats(address(dist)).firstBuyStamp, 0,
"distributor history deleted");
}

```

Giving the payee an identity of their own before step 2 produces the other outcome: the distributor's NFT is burned instead of transferred, and the payee's own `lastBuyAmount` is overwritten with zero.

Recommendations

Replace the code size heuristic with an explicit registry of forwarder addresses maintained by the deployer, and never move an identity NFT or delete `UserStats` for an address that has not opted in.

Note that code size does not distinguish a router from a user wallet. Under EIP-7702 a delegated externally owned account also carries code, so a heuristic based on `to.code.length` will flag ordinary user wallets as well as contracts.

[M-04] Reward claims do not refresh balance-based NFT traits

Severity

Impact:

Low

Likelihood:

High

Description

The project is built around an ERC721 identity whose metadata evolves from the holder's ERC20 activity. Balance is one of the main inputs: `Weapon` and `Border` are both computed from `userStats.lastTokenAmount`.

```

traits[2] = Trait(
    2,
    uint8(
        computeWeaponValue(
            stats.firstBuyAmount,
            stats.lastTokenAmount,
            uint128(totalSupply)
        )
    )
);

traits[4] = Trait(
    4,
    uint8(
        computeBorderValue(stats.lastTokenAmount, uint128(totalSupply))
    )
);

```

`claimReward` pays the holder by increasing their ERC20 balance:

```

rewardPot -= rewardAmount;
hasClaimedReward[tokenId] = true;
normalTransfer(address(this), nftOwner, rewardAmount);

```

`normalTransfer` only moves balances and emits the ERC20 event. It does not call `triggerUpdate`, and `claimReward` does not write a new `userStats[nftOwner].lastTokenAmount`.

```

function normalTransfer(address from, address to, uint value) internal {
    require(balanceOf(from) >= value, "balance too low");
    balances[to] += value;
    balances[from] -= value;
    emit Transfer(from, to, value);
}

```

The reward can be large. With the current supply, the per-claim cap is `totalSupply / 400`, or `25,000e18`. A holder can receive that payout, but their identity NFT still computes `Weapon` and `Border` from the old recorded balance.

For example, a holder with `10,000e18` recorded tokens can claim a `10,000e18` reward. Their real ERC20 balance becomes `20,000e18`, but `lastTokenAmount` stays `10,000e18`, so the NFT continues to render the pre-reward balance tier.

This breaks the main metadata lifecycle at the reward step. The protocol pays the holder for reaching the final character trait, but the same transaction leaves the holder's evolving identity behind the actual position that the protocol just created.

Recommendation

Refresh the holder's recorded balance after paying the reward:

```
normalTransfer(address(this), nftOwner, rewardAmount);
userStats[nftOwner].lastTokenAmount = uint128(balances[nftOwner]);
```

[M-05] Attacker Can Corrupt A Victim's Trading Stats Using A Stale Forwarder

Severity

Impact:

Medium

Likelihood:

Medium

Description

`isForwarder[addr]` is set once a contract buys directly from the pair, but it's only cleared inside the identity-merge branch that actually fires. Three other paths leave it stuck at `true`: the "small transfer, acts like a normal holder" escape (only a local variable resets, not storage), a forwarder selling straight back to the pair, and `transferByNFTContract` (which wipes the contract's balance and stats but never touches this flag).

An attacker doesn't need this to happen by chance; they can create and bank it deliberately: deploy a forwarder, use it once, then strike with it whenever they choose.

This only lands on a victim who has sold at least once and then rebought (`lastSellStamp != 0` and `lastBuyStamp > lastSellStamp`). `getDaysHeld` only reads `lastBuyStamp` in that state; for a holder who has never sold, it computes age from `firstBuyStamp` alone and ignores `lastBuyStamp` entirely, so overwriting it there would do nothing.

Proof of Concept

1. Alice sold once ~100 days ago and rebought an hour later, so her real holding age is computed from that rebuy and reads as roughly 100 days.
2. Bob deploys `R` and buys 100e18 tokens through it (`isForwarder[R] = true`, minted NFT #5).
3. He moves `R`'s NFT away. `R`'s balance and stats get swept, but `isForwarder[R]` stays `true` in storage: a banked, reusable flag that costs nothing to hold indefinitely.
4. Any time later, Bob sends `R` 1,000e18 tokens from another wallet, then has `R` forward 950e18 to Alice.

5. Because `isForwarder[R]` is still `true`, `triggerUpdate` treats this as an identity hop and overwrites Alice's `lastBuyStamp` with R's, which is 0 since R's stats were deleted in step 3. Her `lastSellStamp` (100 days ago) is now no longer before her `lastBuyStamp`, so `getDaysHeld` collapses her holding age to 0, at a time and target entirely of Bob's choosing.

Recommendations

It is recommended to clear `isForwarder[from]` in every path that invalidates the address's forwarder-relevant stats, not only the merge branch.

```
if (!_isForwarder) {
    uint128 lastTokenAmount = userStats[from].lastTokenAmount;
    if (balances[from] >= lastTokenAmount / 10) {
        userStats[from].lastTokenAmount = uint128(balances[from]);
        _isForwarder = false;
+       isForwarder[from] = false;
    }
}
```

```
function transferByNFTContract(address from, address to) external {
    require(msg.sender == address(nftContract), "only NFT contract can
call this function");
+   isForwarder[from] = false;
    uint transferAmount = balanceOf(from);
    ...
}
```

[M-06] Forwarder-hop `lastBuyStamp` overwrite uses the forwarder's own buy timestamp instead of `block.timestamp`, letting to's holding-streak clock silently move backward

Severity

Impact:

Medium

Likelihood:

Medium

Description

When a forwarder relays $\geq 20\%$ of to's balance to an existing NFT holder, triggerUpdate refreshes to.lastBuyStamp from from's stored value rather than the current block time:

```
if (value >= (userStats[to].lastTokenAmount * 20) / 100) {
    userStats[to].lastBuyStamp = userStats[from].lastBuyStamp;
}
```

No check exists to ensure the inherited value is even more recent than what to already had the overwrite is unconditional. This can move to's effective "last buy" backward in time relative to their own history, with a direct consequence on DurationConverter.getDaysHeld:

```
if (lastSellStamp >= lastBuyStamp) return 0;
```

Concrete scenario: to sells at day 50 (lastSellStamp = 50). At day 60 he buys some amount so its last buy stamp is set to 60 then at day 70, he receives a large, genuine forwarded inflow but the relaying forwarder's own lastBuyStamp is day 10 (or 0). to.lastBuyStamp is set to 10. getDaysHeld then evaluates $50 \geq 10$ as true and returns 0 days held, even though to just received real value at day 60. This corrupts Sigil, Character, and via isCharacter6 claimReward eligibility, for a real economic event, based entirely on which forwarder happened to relay the funds.

The user has to now again buy some amount so as to restart its token held timestamps.

Recommendations

Use the current block time, which is always correct and requires no assumption about the forwarder's own history:

```
userStats[to].lastBuyStamp = uint128(block.timestamp)
```

[M-07] Forwarder's small-transfer threshold uses a self-resetting baseline, letting chunked transfers drain a forwarder without triggering identity resolution

Severity

Impact:

Medium

Likelihood:

Medium

Description

`triggerUpdate` classifies a forwarder transfer as "small" (no identity transfer) or "large" (full handoff) by comparing the post-transfer balance to 10% of `lastTokenAmount` but overwrites that baseline to the current balance on every "small" classification:

```
uint128 lastTokenAmount = userStats[from].lastTokenAmount;
if (balances[from] >= lastTokenAmount / 10) {
    userStats[from].lastTokenAmount = uint128(balances[from]);
    _isForwarder = false;
}
```

Since the baseline resets each call, splitting one large forward into repeated ~50% transfers keeps every individual transfer under the threshold relative to the shrinking baseline, even though cumulative outflow can drain ~99%+ of the original balance. `forwarderTransfer` and the full stats handoff never fire, so recipients get real tokens but no NFT/identity reproducing #5 failure mode via chunking. Whoever receives the final leftover chunk inherits the forwarder's entire history despite holding almost none of the value.

Recommendations

Anchor the threshold to a fixed snapshot (balance at first receipt from the pair) and compare cumulative outflow against it, rather than re-deriving 10% from the balance remaining after each transfer.

[L-01] A one wei payment permanently forfeits CHARACTER5, a trait defined as "hasn't sold"

Description

In the branch `Token.triggerUpdate` labels as a normal transfer, the sender is put through the sell path:

```
} else {
    //If the transfer is to another user, update receiver last recorded
    balance
    updateUserStatsOnSell(from, uint128(value));
    userStats[to].lastTokenAmount = uint128(balances[to]);
}
```

The branch comment above reads `Normal transfer, not buy or sell`, and the inner comment describes the goal as refreshing the receiver's balance. Only `lastTokenAmount` serves that purpose. The call also applies full sell semantics to the sender, setting `lastSellStamp` to now, incrementing `sellStreak` and clearing `buyStreak`.

`isCharacter5` is described in its own comment as a holder who bought within the first 72 hours and has not sold since, and it implements that last condition as `lastSellStamp == 0`. Any outgoing transfer of

any size satisfies the condition, so an early buyer who pays a friend one wei forfeits the trait. Nothing ever returns `lastSellStamp` to zero for that holder, so it cannot be re-earned however long they continue to hold.

The exposed population is narrower than that comment suggests. `isCharacter5` additionally requires `tokensHeld * 10000 >= totalSupply * 50`, or 0.5% of supply, which is 50,000e18 at the current supply and is not mentioned in the comment at all. Only holders above that bar could hold the trait in the first place, so only they can lose it this way.

The size of the movement is recorded in `lastSellAmount`, which no on chain code reads, so the classification cannot distinguish a one wei payment from a full exit.

The same stamp also zeroes `getDaysHeld`, removing reward eligibility and the Sigil, but that is recoverable by buying again and the zeroing appears deliberate for genuine sells. The open question is only whether an ordinary transfer should count as one.

Nothing signals any of this. There is no revert and no distinct event, only the usual `Transfer`, so a holder discovers the loss when their artwork changes or a claim fails.

Recommendations

Treat a movement as a sell only when it is significant relative to the sender's balance before the transfer, reusing the shape already present in the forwarder hand off, `balances[from] >= lastTokenAmount / 10`, or reserve sell semantics for transfers that reach the pair. Keep refreshing `lastTokenAmount` on every transfer either way so the balance based gates still see a reduced position.

If the current behaviour is intended, acknowledge it and document it prominently instead: state plainly that any outgoing transfer, including a single wei to another wallet, permanently forfeits CHARACTER5 and resets days held. Holders have no way to learn this from the contract itself.

[L-02] Anyone can force-mint an identity NFT onto any address, permanently blocking every identity transfer into it

Description

`MinimalERC721.transferFrom` rejects any recipient that already holds an identity, before any ERC721 logic runs:

```
require(balanceOf(to) == 0, "recipient already owns a token");
super.transferFrom(from, to, tokenId);
```

The identity that triggers that rejection can be placed on any address by anyone.

`Token.triggerUpdate` selects its buy branch from the transfer's sender alone, never from who initiated it, and the mint sits outside the code-size test so it runs for externally owned accounts as well as contracts:

```

if (from == uniswapV2Pair) {
    if (to.code.length > 0) {
        isForwarder[to] = true;
        userStats[to].lastTokenAmount = uint128(balances[to]);
    }
    updateUserStatsOnBuy(to, uint128(value));
}

```

`updateUserStatsOnBuy` calls `nftContract.mint(user)` whenever the recipient has no recorded purchase. Reaching it costs nothing: `UniswapV2Pair.skim(to)` takes an arbitrary `to`, is callable by anyone, and on a synced pool moves zero tokens — it is exactly `Token.transfer(to, 0)` originating from the pair. `MinimalERC721.mint` uses `_mint` rather than `_safeMint`, so the recipient has no receiver hook and cannot refuse. The attacker needs no tokens, no allowance, no ERC721 rights, and no interaction with the victim.

From then on every `transferFrom` into that address reverts. A marketplace fill, an escrow release, an OTC settlement and an airdrop all fail with `recipient already owns a token`.

This is persistent state rather than a race, so no mempool visibility or ordering advantage is required. Marking a target once is enough, and the address stays blocked until its holder intervenes.

The exposed set is the set of legitimate recipients. To receive an identity an address must have `balanceOf == 0`, and in this system that condition almost always coincides with `firstBuyStamp == 0`, which is precisely what the mint requires. An address that never bought qualifies, and so does one whose identity already left — both `transferByNFTContract` and the forwarder hand-off run `deleteUserStats[from]`, restoring the address to a mintable state.

Escape is possible but expensive and can be repeated against. Transferring the unwanted identity out calls `Token.transferByNFTContract`, which moves the holder's **entire** ERC20 balance to the new NFT owner and deletes their `UserStats`, including any accrued `firstBuyStamp`. There is no partial option, and no renounce or burn function the holder can call. Once the address is clear it can be marked again for the price of another skim, so the attacker pays gas per round while the victim pays a full position relocation and their accrued history.

Recommendations

Mint an identity on a pair-sourced receipt only when the recipient initiated the swap or has explicitly opted in, so that a third party cannot choose who receives one.

Add a renounce or burn path so an unwanted identity can be discarded without dragging the holder's ERC20 balance and history along with it.

[L-03] `getDaysHeld` and `getDaysHeldInHours` implement the same rule with different comparison operators

Description

`DurationConverter` provides two duration helpers that express the same rule, and they disagree on the boundary case.

`getDaysHeld` (token.sol:36) treats a sell that lands on the same timestamp as the last buy as disqualifying:

```
//If the last sell is after the last buy, then return 0 days held, as the
user has sold tokens and has not bought after that
if (lastSellStamp >= lastBuyStamp) {
    return 0;
} else {
    return fromDurationToDays(uint128(block.timestamp) -
uint128(lastBuyStamp));
}
```

`getDaysHeldInHours` (token.sol:63) carries the identical comment and the opposite operator:

```
//If the last sell is after the last buy, then return 0 days held, as the
user has sold tokens and has not bought after that
if (lastSellStamp > lastBuyStamp) {
    return 0;
} else {
    return fromDurationToHours(uint128(block.timestamp) -
uint128(lastBuyStamp));
}
```

The shared comment describes the condition as the last sell being *after* the last buy, which is strictly greater. Only the second function implements that. For `lastSellStamp == lastBuyStamp` the two return contradictory answers on identical state: `getDaysHeld` reports zero duration, while `getDaysHeldInHours` skips the guard entirely and measures the full elapsed time from `lastBuyStamp`.

That state is ordinary. `Token.triggerUpdate` stamps `lastSellStamp` on any outgoing peer transfer, not only on a sale into the pair, so any address that receives a pair-sourced transfer and sends tokens onward in the same block satisfies it.

The divergence is latent today, since `getDaysHeldInHours` has no call site and appears exactly once in `src/` as its own declaration. The risk is that nothing marks the two as differing: a future caller wired to the hours variant would silently disagree with the live day-based eligibility gate on identical state, over a single character with no test, comment or naming difference to signal it. `DurationConverter` is an all-`internal` library inlined into both `Token` and `Traits`, and `Traits` is created in the `Token` constructor with no setter, so whichever semantics ship are fixed for the life of the deployment.

Recommendations

Decide which reading is intended and apply it to both. If a same-block buy and sell should count as having sold, `>=` is correct and `getDaysHeldInHours` needs changing; if the rule is strictly "the sell came after the buy", `>` is correct and `getDaysHeld` does. Update the shared comment to match, since it currently describes only one of the two.

[L-04] `supportsInterface` advertises ERC-4906 but `MetadataUpdate` is never declared or emitted

Description

`MinimalERC721.supportsInterface` returns true for `0x49064906`. [ERC-4906](#) assigns that identifier to an interface declaring two events:

```
interface IERC4906 is IERC165, IERC721 {
    event MetadataUpdate(uint256 _tokenId);
    event BatchMetadataUpdate(uint256 _fromTokenId, uint256 _toTokenId);
}
```

The contract declares only `BatchMetadataUpdate`, and declares it as a local event rather than by inheriting `IERC4906`, so nothing forces the second event to exist. `MetadataUpdate(uint256)` appears nowhere in the codebase and is never emitted.

The contract does emit `BatchMetadataUpdate` over the full id range, so no metadata change goes unsignalled. The defect is that `supportsInterface` answers yes to a question the contract does not support: an integrator that checks `0x49064906` and filters on `MetadataUpdate(uint256)` gets a subscription that never fires, and the event is absent from the ABI entirely.

It also costs precision. Traits here derive from each holder's own `userStats`, so a given transfer affects at most the two identities involved, yet the contract can only ever ask consumers to refresh the entire collection. That is the most expensive request an indexer can be given, and it never says which token actually changed. Two of the three call sites, `forwarderTransfer` and `transferFrom`, already know which id changed and could name it; the third and busiest, `Token.triggerUpdate`, calls `triggerMetadataUpdate()` with no arguments and does not.

Recommendations

Inherit `IERC4906` so `MetadataUpdate(uint256)` is present in the ABI, using the base order `contract MinimalERC721 is IERC4906, ERC721` and keeping the `bytes4(0x49064906)` literal in `supportsInterface`, since `type(IERC4906).interfaceId` evaluates to zero. Continuing to emit only `BatchMetadataUpdate` is compliant, as OpenZeppelin's own `ERC721URIStorage` declares both events and emits only the other one, and it suits this collection, where several traits advance with elapsed time and so have no transaction to signal against.

[L-05] `totalSupply` returns a mint counter that is

never decremented when an NFT is burned

Description

`MinimalERC721.totalSupply` returns `nextTokenId`, which only ever increases:

```
function totalSupply() public view returns (uint256) {
    return nextTokenId;
}
```

`nextTokenId` is incremented on every mint and nothing lowers it. `forwarderTransfer` burns an identity with `_burn(tokenId)` whenever the recipient already owns one, so after any such burn the reported count is higher than the number of NFTs that actually exist, and it stays wrong permanently.

[ERC-721](#) specifies `totalSupply` as "a count of valid NFTs tracked by this contract, where each one of them has an assigned and queryable owner not equal to the zero address". A burned token has no owner, so counting it contradicts the meaning integrators attach to this function name and signature. Anything that walks 1 through `totalSupply()` also breaks, because `tokenURI` calls `_requireOwned` and reverts on a burned id.

Recommendations

Track burns in a separate counter and return `nextTokenId` minus that counter. Do not decrement `nextTokenId` itself, since it assigns the next id and lowering it would hand out an id that was already used.

[L-06] `Border` and `Background` trait values do not increase with rarity

Description

`computeBorderValue` returns enum values that do not follow the rarity order of the thresholds selecting them:

```
tokensHeld * 100000 >= totalSupply * 500 -> Border.BORDER1    // >= 0.5%,
Diamond
tokensHeld * 100000 >= totalSupply * 100  -> Border.BORDER4    // >= 0.1%,
Plat
tokensHeld * 100000 >= totalSupply * 50    -> Border.BORDER2    // >= 0.05%,
Gold
tokensHeld * 100000 >= totalSupply * 25    -> Border.BORDER3    // >= 0.025%
```

Read from rarest to commonest the values run 1, 4, 2, 3. `computeBackgroundValue` has the same problem on its time windows: earliest to latest gives `BACKGROUND3` for the first 72 hours, then

`BACKGROUND1`, `BACKGROUND2`, `BACKGROUND4` and `BACKGROUND5`, so the values run 3, 1, 2, 4, 5.

The contract exposes traits as `struct Trait { uint8 traitId; uint8 traitValue; }`, bare integers with no names attached, so a consumer that sorts or filters on the value ranks holders incorrectly. The rarest Border shows as tier 1 while a more common one shows as tier 4. Whether this matters depends on how the metadata server interprets the number, which cannot be determined from the contract alone. The comments do use ranked names, with Diamond above Plat above Gold, which suggests the values were meant to read as ranks.

This needs to be settled before deployment. The `Traits` contract is created in the `Token` constructor with no setter, and the mappings are hardcoded in `pure` functions, so the numbering cannot be changed on a deployed instance.

Recommendations

Renumber the `Border` and `Background` enum returns so the value increases with rarity. If the numbers are art asset ids and the ordering is deliberate, keep them as they are but state that clearly in the code comments and in the published collection metadata, so integrators and rarity tools do not treat `traitValue` as a rank.

[L-07] Redundant `>= 90 days` branch in `computeBackgroundValue` contradicts its own comment

Description

```
} else if (timeSinceFirstBuy <= 60 days) {
    return Background.BACKGROUND4;
} else if (timeSinceFirstBuy >= 90 days) {
    //Day 60+ buyer is BACKGROUND5
    return Background.BACKGROUND5;
}

return Background.BACKGROUND5;
```

A first buy between 60 and 90 days after launch fails the `>= 90 days` test and falls through to the final return, which yields the same `BACKGROUND5`. The branch cannot change the result for any input.

The comment says "Day 60+" while the condition tests `>= 90 days`, so one of the two is wrong. `Background` declares five members and all five are already assigned, so there is no separate tier the 60 to 90 day window was meant to receive. Either the branch is surplus, or a sixth tier was intended and never added.

Recommendations

Drop the condition and let the final return handle everything past 60 days.

[L-08] The Uniswap pair can be made a reward claimant and its payout is extractable by anyone

Description

`Token.triggerUpdate` treats any transfer sent by the pair as a buy and never excludes the pair as recipient. `UniswapV2Pair.skim(to)` takes an arbitrary `to` and is callable by anyone, so `pair.skim(address(pair))` reaches `updateUserStatsOnBuy(pair, 0)`, which mints the pair an identity NFT and records its size as the pool's entire reserve. `lastSellStamp` can never be set for the pair, because reaching `updateUserStatsOnSell` requires the sender not to be the pair, so its days held grows from `firstBuyStamp` indefinitely. The pair becomes the most eligible claimant in the system.

When the owner claims for it, `claimReward` pays out with `normalTransfer(address(this), nftOwner, rewardAmount)`, which moves tokens into the pool without calling `sync()`. The reward lands as unaccounted surplus and any caller takes it with `pair.skim(attacker)`. If nobody does, the next swap folds it into the reserves as a windfall for liquidity providers. Either way `totalSupply / 400`, or 25,000e18, leaves the reward escrow without reaching a holder.

The amount is bounded to a single occurrence, since `hasClaimedReward` is keyed per tokenId and the pair can neither be minted a second identity nor dispose of the one it has.

Recommendations

Exclude `uniswapV2Pair`, `address(this)` and the NFT contract from the buy branch of `triggerUpdate`, and reject those addresses as `nftOwner` in `claimReward`.

[L-09] Forwarder-delivered buys preserve `sellStreak` while incrementing `buyStreak`

Description

The normal buy path clears the holder's sell streak:

```
function updateUserStatsOnBuy(address user, uint128 amount) internal {
    UserStats storage stats = userStats[user];
    stats.lastBuyStamp = uint128(block.timestamp);
    stats.lastBuyAmount = amount;
    ...
    stats.lastTokenAmount = uint128(balances[user]);

    //Reset Sell Streak on Buy
    stats.sellStreak = 0;

    if (stats.buyStreak < 255) {
        stats.buyStreak += 1;
    }
}
```

```
}  
}
```

The existing-recipient forwarder branch handles a received forwarder balance like a buy for several fields, but does not clear `sellStreak`:

```
if (tokenId != 0) {  
    if (value >= (userStats[to].lastTokenAmount * 20) / 100) {  
        userStats[to].lastBuyStamp = userStats[from].lastBuyStamp;  
    }  
  
    userStats[to].lastBuyAmount = userStats[from].lastBuyAmount;  
    userStats[to].lastTokenAmount = uint128(balances[to]);  
  
    if (userStats[to].buyStreak < 255) {  
        userStats[to].buyStreak += 1;  
    }  
  
    delete userStats[from];  
}
```

As a result, a holder who previously sold can receive a forwarder-delivered position and end up with both a non-zero `sellStreak` and an increased `buyStreak`.

This is inconsistent with the normal buy path, where every buy resets `sellStreak`. Since `Traits.computeEffectValue()` reads `sellStreak`, the holder can keep `Effect1` or `Effect2` even after this buy-like receipt.

Recommendation

Clear `sellStreak` in the existing-recipient forwarder branch when incrementing `buyStreak`:

```
userStats[to].sellStreak = 0;
```

If forwarder-delivered receipts are not meant to count as buys for streak purposes, then `buyStreak` should not be incremented in this branch either.

[L-10] Any User Can Fabricate Their Own Trading History For Free

Description

`Token.transferFrom` was patched to require `value > 0` after a prior finding showed that a zero-value call could rewrite any victim's `UserStats` without a real allowance. The sibling entry point `Token.transfer()` never received the same guard:

```
function transfer(address to, uint value) public returns (bool) {
    normalTransfer(msg.sender, to, value);
    triggerUpdate(msg.sender, to, value);
    return true;
}
```

Since `from` is always `msg.sender` here, this can't be used against another victim. But any wallet, even one holding zero tokens, can call `transfer(uniswapV2Pair, 0)` twice to raise its own `sellStreak` to `2` for free, earning the rarest "sold the most" Effect trait without ever selling a real token. The same call also resets `lastSellStamp`, letting a caller clear their own Character eligibility checks at will.

Recommendations

It is recommended to:

- Mirror the guard already applied to `transferFrom`.

OR

- Keep the zero-value transfer itself working (so Transfer still fires and nothing reverts), and only skip the stats-mutation path:

```
function transfer(address to, uint value) public returns (bool) {
+   require(value > 0, "transfer amount must be greater than 0");
    normalTransfer(msg.sender, to, value);
    triggerUpdate(msg.sender, to, value);
    return true;
}
```

[L-11] Any User Can Erase Their Own Effect Trait With A Dust Buy

Description

The contract's own header comment states "Effect is based on how many times the user has sold the token," but `computeEffectValue` is fed `sellStreak`, a counter that `updateUserStatsOnBuy` unconditionally zeroes on every buy, however small:

```
function updateUserStatsOnBuy(address user, uint128 amount) internal {
    ...
    stats.sellStreak = 0;
    ...
}
```

A wallet that sold 10 times reaches `Effect.EFFECT2` (the rarest tier), but a single 1-wei buy from the pair immediately after resets it to `Effect.NONE`, decoupling the trait from the lifetime activity it claims to represent. The reverse gap exists too ([L-07]): a credit arriving via a forwarder hop instead of a direct buy never resets this streak at all, so the same field is both too easy to erase and, on a different path, fails to update when it should.

Recommendations

It is recommended to track a monotonically increasing lifetime sell counter separate from the consecutive-sell streak, and feed that into `computeEffectValue`.

```
function updateUserStatsOnSell(address user, uint128 amount) internal {
    UserStats storage stats = userStats[user];
    stats.lastSellStamp = uint128(block.timestamp);
    stats.lastSellAmount = amount;

    if (stats.sellStreak < 255) {
        stats.sellStreak += 1;
    }

+   if (stats.totalSells < 255) {
+       stats.totalSells += 1;
+   }

    stats.buyStreak = 0;
    stats.lastTokenAmount = uint128(balances[user]);
}
```

[L-12] SpeedMultiplier Inconsistently Accelerates Trait Thresholds

Description

`speedMultiplier` accelerates `computeSigilValue`, `isCharacter6`, and `isCharacter2`'s duration checks, but `computeBackgroundValue` and `isCharacter5` deliberately ignore it, per their own comments ("based on absolute time since launch, not user behavior"). The same deployment-time knob speeds up half the trait system and leaves the other half untouched. Defensible per-trait, but inconsistent overall, and it can produce visibly incoherent combinations (e.g. at `speedMultiplier = 3`, Sigil tiers evolve 3x faster than the Background tier they're meant to pair with).

Recommendations

It is recommended to document explicitly, in one place, which traits are launch-anchored (Background, Character5) versus behavior-anchored (Sigil, Character6, Character2), and confirm this split is the intended design before shipping; if not intended, apply `speedMultiplier` consistently across all duration-based traits.

```

function computeBackgroundValue(
    uint128 firstBuyStamp,
    uint128 launchTimestamp
) public view returns (Background) {
    uint128 timeSinceFirstBuy = firstBuyStamp - launchTimestamp;

-   if (timeSinceFirstBuy <= 72 hours) {
+   if (timeSinceFirstBuy <= (72 hours) / speedMultiplier) {
        return Background.BACKGROUND3;

-   } else if (timeSinceFirstBuy <= 15 days) {
+   } else if (timeSinceFirstBuy <= (15 days) / speedMultiplier) {
        return Background.BACKGROUND1;
    ...

```

[L-13] QuoteTokensInWETH Returns A Stale, Uncorrectable Price Quote

Description

`quoteTokensInWETH` hardcodes `UNISWAP_V2_ROUTER` and silently returns `0` on any revert from `getAmountsOut`:

```

try IUniswapV2Router02(UNISWAP_V2_ROUTER).getAmountsOut(tokenAmount, path)
returns (uint[] memory amounts) {
    wethAmount = amounts[1];
} catch {
    wethAmount = 0;
}

```

There's no check that this hardcoded router's canonical pair matches the actual `uniswapV2Pair` the owner configured, and any failure (no liquidity, pair not created yet) degrades to a misleading "worthless" quote instead of signaling failure. This function isn't called from `nft.sol`'s on-chain metadata builder; it's purely a display helper for off-chain callers, so it can't move funds. The practical impact is limited to a misleading number if a frontend consumes it directly.

Recommendations

It is recommended to derive the router/pair address from `uniswapV2Pair` (or validate it matches) rather than a separate hardcoded constant, and to have off-chain consumers treat a `0` return as "unknown," not "worthless."

```

function quoteTokensInWETH(
    uint256 tokenAmount
- ) external view returns (uint256 wethAmount) {

```

```

+ ) external view returns (uint256 wethAmount, bool valid) {
    if (uniswapV2Pair == address(0) || tokenAmount == 0) {
-         return 0;
+         return (0, false);
    }

    ...

    try IUniswapV2Router02(UNISWAP_V2_ROUTER).getAmountsOut(tokenAmount,
path)
    returns (uint[] memory amounts) {
        wethAmount = amounts[1];
+         valid = true;
    } catch {
        wethAmount = 0;
+         valid = false;
    }
}

```

[L-14] `isCharacter6` incorrectly checks for token to be held for more than 90 days

Description

```

//Character6 is a user that has held more than 0.1% of the supply for more
than 90 days
if (
    tokensHeld * 10000 >= totalSupply * 10 &&
    daysHeld >= 90 / speedMultiplier
) {

```

The comment states the holding requirement as a flat "more than 90 days," but the enforced condition is `90 / speedMultiplier`, which resolves to 45 days at `speedMultiplier = 2` or 30 days at `speedMultiplier = 3`. As this function is important for reward distribution so if enough days haven't passed this incorrect check will allow incorrect rewards distribution.

Recommendations

Omit usage of speed multiplier in this function.

[I-01] Transfers to the zero address do not reduce `totalSupply`

Description

The token transfer code accepts `address(0)` as a recipient:

```
function normalTransfer(address from, address to, uint value) internal {
    require(balanceOf(from) >= value, "balance too low");
    balances[to] += value;
    balances[from] -= value;
    emit Transfer(from, to, value);
}
```

Both `transfer()` (`token.sol:194–198`) and `transferFrom()` (`token.sol:382–396`) use this function without checking `to != address(0)`.

Tokens sent there are permanently inaccessible, but `totalSupply` stays at `10_000_000e18`.

This makes the reported supply higher than the amount held by live accounts. An approved ERC-20 spender can also burn the approved amount by calling:

```
token.transferFrom(holder, address(0), amount);
```

The spender still needs an allowance, so this is not an arbitrary-burn bug. The problem is that a burn-like action is available through the normal transfer interface without updating supply.

Indexers and integrations that rely on `totalSupply` may report incorrect circulating supply. The fixed-supply percentage checks in `Traits` can also differ from the actual amount of tokens still in circulation.

Recommendation

Reject the zero address in `normalTransfer()`:

```
function normalTransfer(address from, address to, uint value) internal {
    require(to != address(0), "transfer to zero address");
    require(balanceOf(from) >= value, "balance too low");
    balances[to] += value;
    balances[from] -= value;
    emit Transfer(from, to, value);
}
```

If burning is intended, add a separate `burn()` function that decreases `totalSupply`.